Single-Core Code Optimization

Lecture notes — HPC 1 (DSAI + SDIC), Università di Trieste, a.y. 2025/2026

These notes accompany the slides "Code Optimization — Many Fundamental Techniques" and expand on them considerably. They are meant to be read after the lectures, not instead of them; several points will be clearer if you have heard the live discussion first. The example codes referred to throughout are in the course repository under `sco/` *(Single-Core Optimization).*

Cross-references to other lecture notes are given where relevant: "cache_policies_impact.pdf", "matrix_transpose---cache_oblivious.pdf", "hot_cold_fields.pdf", "linked_lists_are_a_bad_choice_in_hpc.pdf". Hennessy & Patterson, "Computer Architecture" (7th ed., 2025) is the primary textbook reference.

Part I — Cache and Memory Hierarchy

1. The memory wall, once more

We have already discussed the memory wall at length in the architecture lectures, so let us just recall the essential point. The performance gap between the CPU's ability to issue memory requests and the DRAM's ability to respond has been growing since the mid-1980s and has never been closed; the Hennessy & Patterson figure that plots relative performance with 1980 as baseline makes this visually obvious, even if it does not include HBM (which, for our purposes, mitigates the bandwidth problem but does not remove the latency problem).

A modern core running at 3–4 GHz completes a cycle in 0.25–0.33 ns. A round trip to DRAM costs 50–100 ns, which is 150–300 clock cycles. If the CPU issues a load and the data is not in any cache, the pipeline stalls: the instruction cannot retire, everything that depends on it blocks, and eventually the reorder buffer fills up and the core just sits there, waiting for DRAM.

This is not a pathological scenario. It is the *default* for any code that accesses data in a non-trivial pattern — a tree traversal, a hash table lookup, a sparse matrix-vector product. For such codes, useful arithmetic accounts for a small fraction of the total runtime; the rest is waiting.

2. The cache hierarchy — a quick recap

The engineering answer to the memory wall is the cache hierarchy: progressively larger, progressively slower memories between the core and DRAM. The technology is different — SRAM rather than DRAM — which makes it faster, more power-hungry, and far more expensive per bit.

On a typical modern x86 CPU:

- **L1**: 32–48 KB per core (data) + 32–48 KB (instructions), ~4 cycle latency. This is *inside* the core.
- **L2**: 256 KB – 2 MB per core, ~12–14 cycle latency. Also inside the core on recent designs (Raptor Lake, Zen 4).
- **L3**: 16–96 MB shared across all cores, ~30–50 cycle latency. Inside the CPU package.
- **DRAM**: tens to hundreds of GB, ~150–300 cycle latency.

The reason caches work at all — rather than being a hopelessly small buffer that constantly evicts what you need — is the *principle of locality*: programs tend to re-use data they have accessed recently (temporal locality) and to access data that is close in address space to what they accessed recently (spatial locality). Instruction streams exhibit this even more strongly than data streams, which is why L1 is split into I-cache and D-cache.

The cache does not operate on individual bytes. The unit of transfer between levels is a *cache line*, which on essentially all modern x86 is **64 bytes**. When you access a single `double` (8 bytes), the hardware brings the entire 64-byte line into L1. This is the hardware's bet on spatial locality: you probably need the neighbouring bytes too. If you do, you get them "for free"; if you do not, you have wasted 56 bytes of bandwidth and cache capacity.

This single design choice — the 64-byte line — has profound implications for everything that follows.

3. How DRAM maps into the cache

Here is where things get concrete. The RAM can hold 10^10 bytes or more; L1 holds 3×10^4 bytes. Somehow we must decide, for any given memory address, *where* in the cache it can go. There are three canonical strategies:

Fully associative — a line from memory can go anywhere in the cache. This minimizes conflicts (you only evict something when the cache is full) but makes lookup expensive: to check whether a given address is cached, you must compare against *every* line in the cache. For a 32 KB L1 with 64-byte lines, that is 512 comparisons per access — clearly impractical as a general solution.

Direct-mapped — each memory address maps to exactly one cache line, determined by modular arithmetic on the address. Lookup is trivial (one comparison) but conflicts are maximised: if two frequently-used addresses happen to map to the same line, they evict each other continuously regardless of how much empty cache there is.

Set-associative — the practical compromise. The cache is divided into *sets*, each containing *w* lines (the "ways"). An address maps to exactly one set (by modular arithmetic), but within that set it can go in any of the *w* lines. A typical modern L1 is **8-way set-associative** with 64-byte lines: 32 KB / (64 × 8) = 64 sets.

The set assignment is determined by the address itself. If we decompose the address into its bit fields:

```
[ tag bits | set index (s bits) | line offset (b bits) ]
```

For a 32 KB, 8-way, 64-byte-line L1: *b* = 6 (since 2^6 = 64 bytes per line), *s* = 6 (since 2^6 = 64 sets). The first 6 bits of the address select the byte within the line; the next 6 bits select the set; the remaining bits form the *tag*, which is stored alongside the line so that the cache can reconstruct which memory address the line belongs to.

Remind: the cache *always* reasons in units of lines — 64 bytes. When you read a single byte, you bring in 64 bytes. When you dirty a single byte, you dirty the entire line.

4. The three Cs and the three Rs

Three types of cache misses:

- **Compulsory** (cold misses): the first time you access an address that has never been in the cache. Unavoidable in principle, though prefetching can hide their latency.
- **Capacity**: the working set is larger than the cache; lines must be evicted even if there are no address conflicts.

- **Conflict**: the cache has room, but the address maps to a set that is already full. This is the pathological case: it can occur even when the cache is almost empty, if your access pattern has unfortunate stride properties.

Three strategies to improve cache usage:

- **Rearrange** (code and data): design the layout so that temporally-local accesses are also spatially local — same line, or at least same set neighbourhood.
- **Reduce** (size): smaller data types, fewer instructions, smaller working sets.
- **Reuse** (cache lines): once a line is in cache, extract as much work from it as possible before it is evicted.

5. The memory access pattern — why it is everything

Let us work through three tiny examples to build intuition. Consider a direct-mapped cache with 16 bytes total, 2 lines of 8 bytes each (accommodating 2 floats per line), and a cache-aligned float array `X[8]`.

Example 1: sequential access, two passes.

```
for (int j = 0; j < 2; j++)
    for (int i = 0; i < 8; i++)
        access(X[i]);
```

Hit/miss pattern: `M H M H M H M H  H H H H H H H H`. Miss rate: 50%. All misses are compulsory (first pass); the second pass hits entirely.

Example 2: strided access.

```
for (int j = 0; j < 2; j++) {
    for (int i = 0; i < 7; i += 2) access(X[i]);    // evens
    for (int i = 1; i < 8; i += 2) access(X[i]);    // odds
}
```

Hit/miss pattern: `M M M M  M M M M  M M M M  M M M M`. Miss rate: 100%. The even-indexed elements evict the odd-indexed ones (they map to the same line in this tiny direct-mapped cache), and vice versa. These are all *conflict* misses after the first pass.

Example 3: blocked access.

```
for (int k = 0; k < 2; k++)
    for (int i = 0; i < 2; i++)
        for (int j = 4*k; j < (k+1)*4; j++)
            access(X[j]);
```

Miss rate: 25%. By processing the array in blocks that fit in the cache, we get temporal reuse within each block.

The takeaway is blunt: **the memory access pattern is of primary importance**. The same data, the same number of accesses, three different patterns, and the miss rate ranges from 25% to 100%.

6. The memory mountain

The "memory mountain" is a benchmark that plots effective bandwidth as a function of array size and stride. You will find the code in `examples_on_cache/memory_mountain/mountain.c`.

When the array fits in L1 (say, ≤ 32 KB) and the stride is 1, you hit the peak: on modern hardware, 10–15 GB/s per core for sequential L1 access is typical. As the array grows past L1, L2, and L3 capacities, the effective bandwidth drops — you can see clear "ridges" corresponding to each cache level. The deep valley at large size and large stride is the DRAM floor: ≤ 0.5 GB/s for pointer-chasing access, where every load is a full-latency cache miss.

This plot is worth staring at. It tells you in one picture why sequential access is 20–30× faster than random access for large data, and why fitting your working set in L1 (or at least L2) matters more than almost anything else.

A rough guideline for achievable per-core bandwidths:

Access type	Notes	Effective bandwidth
Sequential, L1	Everything fits	10–15 GB/s
Sequential, L3		5–10 GB/s
Sequential, DRAM		2–5 GB/s
Strided (= cache line)		Above figures / elements_per_line
Random (pointer chasing)	Latency-bound	≤ 0.5 GB/s

These are order-of-magnitude figures and they depend on the specific hardware, but they are useful as a sanity check when you profile your code.

7. Matrix transpose: a case study in strided access

Matrix transposition is the simplest non-trivial example of a memory access pattern problem. You have two N×N matrices in row-major order, and you need B[j][i] = A[i][j]. The naive implementation:

```
for (int i = 0; i < N; i++)
    for (int j = 0; j < N; j++)
        B[j * N + i] = A[i * N + j];
```

Here A is read sequentially (row-major, stride 1) and B is written with stride N. Or you can flip the loops:

```
for (int i = 0; i < N; i++)
    for (int j = 0; j < N; j++)
        B[i * N + j] = A[j * N + i];
```

Now B is written sequentially and A is read with stride N. A strided access on one side is unavoidable; the question is: **is it better to stride on reads or on writes?**

The answer, perhaps counter-intuitively, is that strided reads are generally cheaper than strided writes. The reason lies in the cache write policy.

8. A word on write policies

When you modify a variable in cache, the hardware must eventually propagate that change to DRAM. Two strategies:

Write-through — every write goes to both cache and DRAM immediately. Simple, but floods the memory bus.

Write-back — the write stays only in cache; the line is marked "dirty". The line is flushed to DRAM only when it is evicted. This is what modern caches actually use for most levels, and it is far more bandwidth-efficient — if you write to the same line 100 times, only the final state is written back.

But there is a catch. When you write to an address that is *not* in the cache (a write miss), the hardware must first load the line from memory, modify the relevant bytes, and then hold the dirty line. This is called **write-allocate** (or "read-for-ownership", RFO), and it doubles the memory traffic for write misses: one read (to fetch the line) plus one eventual write (when the dirty line is evicted). The alternative, **no-write-allocate**, writes directly to DRAM without loading the line, which avoids the RFO but does not let the cache benefit from spatial locality on subsequent writes.

Most modern caches combine write-back with write-allocate. This makes strided writes particularly expensive: for a stride larger than one cache line, *every* write is a miss, triggers an RFO (loading 64 bytes that you will mostly not use), dirties the line, and when the line is evicted it must be flushed back — 128 bytes of memory traffic per useful write.

Strided reads, by contrast, trigger a simpler cache miss: load the line, use the one element you need, and when the line is eventually evicted it is *clean* — no writeback needed. That is 64 bytes of traffic, not 128.

This is why, in the matrix transpose, reading A with stride N and writing B sequentially gives better bandwidth than the reverse. The detailed analysis, including a discussion of non-temporal stores and the various policy combinations, is in the separate lecture note `cache_policies_impact.pdf`.

9. Three regimes for the matrix transpose

Let C be the total data-cache size, L_C the line size (64 bytes), S the element size, and N the matrix dimension. We should expect three distinct performance regimes:

Regime 1: $2 \times N^2 \times S < C$. Both matrices fit entirely in the cache. Traversal order is irrelevant; bandwidth is near-maximum.

Regime 2: $N \times L_C < C$, but the full matrices do not fit. A fraction of each column fits in the cache simultaneously, partially alleviating the strided-access penalty.

Regime 3: $N \times S > C \times L_C / S$. Essentially every strided access is a cache miss; performance drops sharply to the DRAM floor.

In practice, the transition between regimes 2 and 3 is where you see the sharpest performance cliffs. The plots you will obtain from `matrix_transpose.c` and `matrix_transpose_swapped.c` show this clearly.

10. The power-of-two trap (cache resonance)

There is a subtlety that catches people. If you run the transpose benchmark for many matrix sizes, you will notice isolated spikes of *terrible* performance at certain specific N values — typically powers of two or small multiples thereof.

This is cache resonance (or cache-associativity conflicts). Recall that the set an address maps to depends on the middle bits of the address. If the matrix dimension N happens to be such that N × S is a power of two and that power of two equals the total set-index range (i.e., N × S ≥ 2^(b+s)), then consecutive elements in a column all map to *the same cache set*. In an 8-way cache, this means that after 8 column elements, every subsequent access is a conflict miss — regardless of how much total cache is available.

The cure is **padding**: instead of allocating an N × N matrix, allocate (N+k) × N where k is a small odd number. This shifts the column stride by k × S bytes, breaking the resonance. In professional numerical libraries, this kind of padding is standard practice; it is cheap (a few extra bytes per row) and the performance recovery can be dramatic.

Remind: there is also a well-known interaction between padding and the compiler's ability to vectorize. When the matrix rows have a power-of-two length, certain pointer aliasing conditions may prevent vectorization entirely; padding with an odd constant can fix both the cache resonance and the aliasing problem simultaneously. The `restrict` qualifier can help resolve the aliasing issue independently; see the codes `matrix_transpose_padded.c` and the discussion in `matrix_transpose---cache_oblivious.pdf`.

11. Enhancing locality: tiling (blocking)

The idea is simple: instead of processing an entire row or column of a matrix in one pass, process small blocks that fit in the cache, so that each element is reused as much as possible before being evicted.

For matrix-matrix multiplication C = A × B, the naive implementation accesses B in column-major order for every element of C, causing catastrophic cache behaviour for large matrices. Tiling partitions the three matrices into small blocks of size (l × h × g); within each block-level operation, the data fits in L1 or L2, and the inner loops achieve excellent temporal reuse.

The expected reduction in cache misses is of the order L / (l × h × g), where L is the cache capacity. With typical block sizes (32–128 elements per side), this can be a factor of 1000 reduction. We will see the actual numbers from the instrumented codes.

The drawback of tiling is that the block size is a tuning parameter: it depends on the cache size, the associativity, the line size, and the specific data types. This is why professional libraries like BLAS tune it per-architecture, and it is also why *cache-oblivious* algorithms — which achieve good locality without any tuning parameter — are of theoretical and practical interest.

12. The Z-order (Morton order) and space-filling curves

Can you get the *effect* of blocking — improved locality for 2D data — without hard-coding a block size?

The Morton order, or Z-order, is one answer. It is based on bit-interleaving: given integer coordinates (x, y), the Morton index is obtained by interleaving the bits of x and y:

```
(x, y) → (y_n x_n ... y_1 x_1 y_0 x_0)
```

For example:

- (0,0) → 0000 = 0
- (1,0) → 0001 = 1
- (0,1) → 0010 = 2
- (1,1) → 0011 = 3

- (2,0) → 0100 = 4 ...

Traversing a 2D array in Morton order produces a Z-shaped recursive pattern that naturally clusters spatially-nearby elements in memory, at every scale — it is a *space-filling curve* (objects discovered by Peano at the end of the XIX century). The key property is that the locality is "automatic": there is no block size to tune, and the curve adapts to whatever cache size the hardware provides.

However — and this is a recurring theme — elegant theory does not guarantee practical performance. The Morton order requires bit manipulation for *every single element access* (to compute the interleaved index), and this per-element overhead can be significant. In practice, a simple recursive divide-and-conquer approach for matrix transposition achieves the same spatial traversal pattern as the Morton order, but without the bit-manipulation overhead. The lesson: be open to changing the traversal order, but measure the cost of the change itself.

The Morton curve only works naturally for power-of-two dimensions, which brings us back to the power-of-two trap. And it requires that the entire array be stored in Morton order, which complicates interfacing with the rest of your code.

For irregular data — particles in 3D space, for instance — space-filling curves (Morton, Hilbert, Peano) are genuinely useful for *data layout optimisation*: sorting particles by their Hilbert index so that spatially-nearby particles are also nearby in memory. This reduces cache misses for neighbour-interaction computations. But for regular dense-matrix operations, explicit tiling with a tuned block size almost always wins.

The details, including worked examples and performance measurements, are in the lecture note `matrix_transpose---cache_oblivious.pdf`.

13. Optimising the data layout: hot and cold fields

The cache hierarchy does not care about your abstractions. It operates on addresses and 64-byte lines. If your `struct` is 320 bytes long and the field you need most is at offset 0 while the field you need second-most is at offset 308, the hardware will need *two separate cache lines* to serve a single logical operation — and 120 bytes of fetched data will be wasted.

The classic example is a linked-list node:

```
struct my_node {
    double   key;            // 8 bytes
    char     my_data[300];   // 300 bytes
    my_node *next_node;      // 8 bytes
};
```

During a search — `while (p != NULL) { if (p->key == key) ...; p = p->next_node; }` — the fields accessed on every iteration are `key` and `next_node`. They are separated by 300 bytes of cold data. This guarantees at least two cache misses per node: one for `key` and one for `next_node`.

First fix: reorder the fields so that `key` and `next_node` are adjacent.

```
struct my_node {
    double   key;            // offset 0
    my_node *next_node;      // offset 8
    char     my_data[300];   // offset 16
};
```

Now `key` and `next_node` are in the same cache line — one cache miss brings both.

Second fix: separate the hot and cold fields entirely.

```
struct my_node {
    double   key;           // 8 bytes
    my_node *next_node;     // 8 bytes
    data_t  *my_data;       // 8 bytes (pointer to cold data)
};
```

The hot part is now 24 bytes. For 10^8 nodes, the hot working set is 2.4 GB instead of ~32 GB. This may make the difference between fitting in L3 and not fitting at all.

The full discussion, including instrumented codes and PAPI counter analysis, is in the lecture note `hot_cold_fields.pdf`.

14. Why linked lists are (almost always) wrong in HPC

Let me just state this bluntly, since it is one of those points where being diplomatic wastes everybody's time: if you are writing performance-sensitive code and you are using a linked list, you are almost certainly making a mistake. The reasons, in order of increasing impact:

1. **Memory overhead**. Each node carries a pointer (8 bytes), plus whatever bookkeeping the allocator maintains. For small payloads, the overhead dominates.
2. **Allocation cost**. A `malloc()` per node is expensive — hundreds of cycles per call, involving kernel-level bookkeeping, and creating fragmentation that compounds over time.
3. **Spatial locality destruction**. Successive nodes can be anywhere in the address space. A traversal through N nodes becomes a random-access pattern across memory, wasting the vast majority of each cache line fetched.
4. **Prefetcher defeat**. The hardware prefetcher detects *regular* strides. Pointer chasing is, by definition, irregular: the address of the next node is unknowable until the current node is fetched. The prefetcher gives up entirely. Every access is a full-latency miss.
5. **TLB pressure**. Each node may reside on a different page. For large numbers of nodes, the TLB (which has a few thousand entries at most) is overwhelmed, adding page-table-walk latency on top of the cache-miss latency.

The alternative is simple: use arrays. Allocate all N nodes contiguously (a single `calloc`), use integer indices instead of pointers. The traversal becomes a sequential or nearly-sequential scan; the prefetcher works; the TLB maps a few large pages and is happy. If you need to insert or delete elements dynamically, maintain a free-list within the array, or use a separate indirection array.

Remind: pointer chasing is even more catastrophic on GPUs, which lack branch predictors and hardware prefetchers entirely and are designed to operate on predictable, contiguous data streams.

A complementary discussion of why linked lists could be a poor choice for HPC in general is in `linked_lists_are_a_bad_choice_in_hpc.pdf`.

Part II — Loops

15. Arithmetic intensity: a framework for thinking about loops

Not all loops are created equal. A useful classification is the **arithmetic intensity** (AI): the ratio of operations performed to the amount of data moved. This tells you whether a loop is *memory-bound* (the CPU spends most of its time waiting for data) or *compute-bound* (the CPU spends most of its time doing arithmetic).

O(N) ops / O(N) data — scanning arrays, 1D vector operations, scalar products. The arithmetic intensity is O(1): one or a few operations per element fetched. These are *inevitably* memory-bound for large N; the only leverage is reducing unnecessary operations and maximising data reuse. Loop fusion is the main tool: if two passes over the same array can be merged into one, you halve the memory traffic.

```
// Before: B is fetched twice
for (int i = 0; i < N; i++) A[i] = B[i] * C[i];
for (int i = 0; i < N; i++) Q[i] = B[i] + D[i];

// After: B is fetched once
for (int i = 0; i < N; i++) {
    A[i] = B[i] * C[i];
    Q[i] = B[i] + D[i];
}
```

$O(N^2)$ ops / $O(N^2)$ data — dense matrix-vector products, matrix transpose, matrix addition. Still O(1) arithmetic intensity per element, but with nested loops there is more room to exploit locality: loop interchange, partial unrolling with outer-loop jam, and tiling.

$O(N^3)$ ops / $O(N^2)$ data — matrix-matrix multiplication, dense matrix diagonalisation. Here the arithmetic intensity is O(N), which means that for large enough N, the computation has *ample* opportunity for data reuse, and performance close to the theoretical peak is achievable — if the loops are structured correctly. This is why MMM is at the core of Linpack and why BLAS libraries are so heavily optimised.

For $O(N^3)/O(N^2)$ problems: it is a brilliant idea to link an optimised library rather than writing your own implementation, unless very special needs must be met.

16. $O(N^2)/O(N^2)$ example: optimising a matrix-vector product

Consider the standard matrix-vector product C[i] += A[i][j] × B[j]:

```
for (int i = 0; i < N; i++)
    for (int j = 0; j < N; j++)
        C[i] += A[i][j] * B[j];
```

This issues ~$3N^2$ memory accesses (reading A, B, and writing C). We can improve it step by step.

Step 1: avoid unnecessary loads/stores. The compiler may not realise that `C[i]` can be held in a register across the inner loop (because of potential aliasing between `C` and `A` or `B`). Explicit scalar replacement fixes this:

```
for (int i = 0; i < N; i++) {
    double c_temp = C[i];
    for (int j = 0; j < N; j++)
        c_temp += A[i][j] * B[j];
    C[i] = c_temp;
}
```

Memory accesses: ~$2N^2 + N$. The saving is modest, but it also exposes the accumulation pattern to the compiler, which may now vectorise the inner loop.

Step 2: unroll the outer loop, fuse into the inner loop. Process m rows of A simultaneously, reusing B[j] across all m rows:

```
for (int i = 0; i < N; i += m)
    for (int j = 0; j < N; j++) {
        double b = B[j];
        C[i]     += A[i][j]     * b;
        C[i+1]   += A[i+1][j]   * b;
        // ... up to C[i+m-1]
    }
```

Memory accesses: ~$N^2(1 + 1/m) + N$. Each element of B is fetched once every m iterations instead of every iteration.

Step 3: tiling. For the matrix-vector case, tiling helps cut TLB misses by accessing 2D arrays in blocks. For the matrix-matrix case (next section), tiling is transformative.

17. $O(N^3)/O(N^2)$ example: matrix-matrix multiplication

The naive implementation:

```
for (int i = 0; i < m; i++)
    for (int k = 0; k < p; k++)
        for (int j = 0; j < n; j++)
            C[i][k] += A[i][j] * B[j][k];
```

For each element of C, we scan a row of A and a column of B. B is accessed in column-major order — stride N — which is catastrophic. The expected cache misses: $mp/L + mnp/L + mnp$, where L is the cache line capacity in elements. The last term — mnp full misses — dominates: we have approximately *one cache miss per floating-point operation*. This is as bad as it gets.

Fix 1: swap inner loops. Putting j before k so that B is accessed in row-major order:

```
for (int i = 0; i < m; i++)
    for (int j = 0; j < n; j++)
        for (int k = 0; k < p; k++)
            C[i][k] += A[i][j] * B[j][k];
```

Now all three arrays are traversed in row-major order. Expected misses: ~$3mnp/L$. The factor-of-L improvement (= 8 for doubles, since a 64-byte line holds 8 doubles) is immediate and significant.

Fix 2: tiling. Partition all three matrices into blocks of size l × h × g chosen so that the three blocks fit in L1 simultaneously. The inner loops operate on block-sized subproblems with excellent temporal reuse; the number of cache misses drops by a factor of approximately L/(l × h × g), which for reasonable block sizes is O(1000).

The tiled code is more complex:

```
for (int ii = 0; ii < m; ii += bsize)
    for (int jj = 0; jj < n; jj += bsize)
        for (int kk = 0; kk < p; kk += bsize)
            // mini-MMM on bsize × bsize blocks
            for (int i = ii; i < min(ii+bsize, m); i++)
                for (int j = jj; j < min(jj+bsize, n); j++)
                    for (int k = kk; k < min(kk+bsize, p); k++)
                        C[i][k] += A[i][j] * B[j][k];
```

The performance results from the instrumented codes (`examples_on_pipelines/matrix_multiplication/`) are dramatic: the tiled version shows drastically reduced L1 misses per element, stable and low cycles-per-element, and high IPC compared to both the naive and loop-swapped versions. This is one of those cases where the effect is so large that there is no ambiguity in the data.

18. Loop unrolling: why it matters more than you think

Loop unrolling is often presented as a minor optimisation — "reduces loop overhead". That is true but uninteresting; the overhead of incrementing a counter and testing a branch is a handful of cycles per iteration, and a modern branch predictor handles the loop branch with near-perfect accuracy.

The real reasons loop unrolling matters are:

1. **It exposes the critical data path.** Compilers reason about dependencies between instructions. Unrolling physically reveals what depends on what, making it easier for the compiler to schedule instructions optimally.
2. **It enables ILP (Instruction-Level Parallelism).** If you have a single accumulator in a reduction loop, every addition depends on the previous one — there is a *serial dependency chain* that limits throughput to one addition per FP-add latency (~4 cycles). Unrolling with multiple accumulators breaks this chain.
3. **It helps with vectorisation.** The compiler needs to see enough independent data to fill a SIMD register (4 doubles for AVX-256, 8 for AVX-512). A tight loop body with a single iteration may not expose that.

Let us trace this through with a simple reduction:

```
for (int i = 0; i < N; i++)
    S += A[i];
```

At `-O3 -march=native`, the compiler generates scalar code for the FP version (double) but fully vectorised code for the integer version. Why? Because floating-point addition is not associative: the compiler is not allowed to reorder the summation, and vectorisation implies reordering. Integer addition *is* associative, so the compiler vectorises freely.

This is not the compiler being stupid. It is being correct. *You* told it the order matters by writing `S += A[i]` with `double S`. If you know the order does not matter for your problem, you have two options:

Option A: tell the compiler with `-fassociative-math`. This gives the compiler permission to reorder FP operations for performance. It is a useful flag, but it is a global relaxation — it affects all FP operations in the translation unit.

Option B: restructure the code yourself.

```
// 2×2 unrolling: two independent accumulators
double s0 = 0.0, s1 = 0.0;
for (int i = 0; i < N - 1; i += 2) {
    s0 += A[i];
    s1 += A[i+1];
}
S = s0 + s1;
```

Now the compiler can see that `s0` and `s1` are independent — each has its own dependency chain, and the two chains can execute in parallel on the CPU's multiple FP units. With 4×4 unrolling (4 iterations, 4 accumulators), the FP throughput reaches near-peak.

Remind: when unrolling by a factor U, always handle the tail:

```
int N_ = (N / U) * U;
for (int i = 0; i < N_; i += U) { /* unrolled body */ }
for (int i = N_; i < N; i++)     { /* scalar cleanup */ }
```

The notation "n×m unrolling" means n iterations are processed per loop step, using m independent accumulators.

19. The compiler flag `-fassociative-math`

A brief but important digression. The `-fassociative-math` flag (GCC; the equivalent exists for Clang and Intel) tells the compiler it may assume floating-point addition and multiplication are associative. This enables:

- Reordering of summations for vectorisation
- Tree-reduction patterns instead of serial chains
- Loop transformations that change operation order

It does *not* enable unsafe things like assuming NaN == NaN or ignoring infinities. It is a much more targeted relaxation than `-ffast-math`, which also enables `-fno-math-errno`, `-funsafe-math-optimizations`, `-ffinite-math-only`, and `-fno-trapping-math` — some of which can change program semantics in ways that are difficult to debug.

My advice: use `-fassociative-math` freely when your algorithm does not depend on a specific summation order (which is most of the time in HPC). Avoid `-ffast-math` unless you understand exactly what each component does and you have verified that none of them breaks your code. A detailed discussion is in the "basic optimization steps" lecture notes.

20. Memory aliasing, `restrict`, and `-fstrict-aliasing`: helping the compiler trust your memory

This section is about a problem that, in my experience, most programmers ignore until it bites them — at which point it bites very hard. It concerns the question: *can the compiler prove that two pointers do not refer to the same memory?* If it cannot, a cascade of missed optimisations follows, sometimes with spectacular consequences.

20.1. What aliasing is and why the compiler cares

Two pointers "alias" when they can refer to overlapping memory. Consider:

```
void add_arrays(double *dst, double *src, int N) {
    for (int i = 0; i < N; i++)
        dst[i] += src[i];
}
```

If the compiler cannot prove that `dst` and `src` do not overlap, it must assume that writing `dst[i]` might change the value of some future `src[j]`. This assumption has consequences that range from inconvenient to devastating:

- **Vectorisation is blocked.** SIMD requires processing multiple elements simultaneously — say, `dst[i..i+3] += src[i..i+3]` in one AVX instruction. But if `dst` and `src` overlap, writing `dst[i]` could change `src[i+1]`, and the result of processing them "in parallel" would differ from processing them sequentially. The compiler must fall back to scalar code. For a loop that could run 4× or 8× faster with AVX, this is a severe penalty.
- **Loads cannot be hoisted past stores.** In a loop that reads and writes through pointers, each store forces the compiler to re-load any value that *might* have been modified. A local variable in a register stays in the register; a value behind a pointer may not.
- **Instruction scheduling is constrained.** The compiler cannot freely reorder loads and stores if they might alias, which limits its ability to fill pipeline slots and hide latencies.

The irony is that in the vast majority of real code, the pointers *do not* alias. You, the programmer, know this perfectly well — you allocated `dst` and `src` from separate buffers and they are nowhere near each other in memory. But the compiler, staring at a function signature with two bare `double *` arguments, has no way to know.

20.2. The `restrict` qualifier

The C99 standard introduced `restrict` precisely for this purpose. When you write:

```
void add_arrays(double * restrict dst, const double * restrict src, int N) {
    for (int i = 0; i < N; i++)
        dst[i] += src[i];
}
```

the `restrict` qualifier is a *contract* between the programmer and the compiler: you are promising that, for the lifetime of each pointer, no other pointer will be used to access the same memory. This is not a check — the compiler does not verify it, and violating the promise is undefined behaviour. But with the promise in place, the compiler can assume that `dst[i]` and `src[j]` never refer to the same location, and it is free to vectorise, reorder, and schedule as aggressively as it likes.

A few practical points:

`restrict` is a C keyword, not (yet) a C++ keyword. In C++ you have to use the compiler extension: `__restrict__` on GCC and Clang, `__restrict` on MSVC and Intel. This is a well-known annoyance. In practice, every production C++ compiler supports some variant of it, and you should not hesitate to use it.

`restrict` is per-pointer, per-scope. You can have some restricted and some unrestricted pointers in the same function. The compiler only uses the information you give it.

`restrict` on function arguments is the most common and useful case. This is where the compiler's alias analysis is weakest, because it has no visibility into how the function is called. Within a function, if you allocate two separate buffers with `malloc`, the compiler usually knows they do not alias (since the C standard guarantees that separate `malloc` calls return non-overlapping memory). But if the pointers arrive as function parameters, or if you derive two pointers from the same allocation (more on this below), the compiler is blind.

20.3. A real-world case study: the matrix transpose debacle

We encountered this in class, and it is worth recounting because it was a genuine surprise — the kind that makes you reconsider what you think you know.

During the Z-order matrix transpose investigation (see `matrix_transpose---cache_oblivious.pdf`), we had two versions of the same algorithm. Version 1 allocated source and destination from a single block:

```
double *src = (double *)malloc(sizeof(double) * size * 2);
double *dst = src + size;
```

Version 2 used two separate allocations:

```
double *src = (double *)aligned_alloc(sizeof(double), sizeof(double) * size);
double *dst = (double *)aligned_alloc(sizeof(double), sizeof(double) * size);
```

Same algorithm, same data, same compiler flags (`-O3 -march=native`). Version 2 ran roughly **5× faster** than version 1.

The reason had nothing to do with the cache — the access patterns were identical. It was entirely a compiler vectorisation decision. In version 1, `src` and `dst` pointed into the same allocation. The Z-order index computation (the `compact_bits` bit-manipulation) was opaque to the compiler, and GCC could not prove that `dst[morton_index_a]` would never coincide with `src[morton_index_b]` for some combination of loop iterations. It gave up and emitted scalar code: ~48 instructions per element, totalling ~200 million instructions.

In version 2, two separate `aligned_alloc` calls guarantee non-overlapping memory. GCC's alias oracle knows this (the C standard says so), and it vectorised the loop with AVX2: ~10 instructions per element amortised, totalling ~40 million instructions.

One `malloc` versus two. A 5× difference in instruction count that is entirely invisible to the algorithm designer and entirely obvious to the compiler.

Adding `restrict` to the version 1 pointers fixed it:

```
double * restrict src = (double *)malloc(sizeof(double) * size * 2);
double * restrict dst = src + size;
```

With this, GCC vectorised version 1 and the instruction counts matched version 2 exactly.

The lesson, which I keep repeating because it keeps surprising people: **memory aliasing is a compiler-visible property**. The decision to allocate from one block versus two had no effect on the algorithm, no effect on the data layout, and no effect on the cache behaviour. It had a 5× effect on the generated code.

20.4. `const` as an aliasing aid

Declaring a pointer `const` does not, by itself, resolve aliasing — the compiler knows you might have a non-const pointer to the same memory elsewhere. However, `const` *does* help in two ways.

First, it communicates intent to the reader and to the compiler. A `const double *src` parameter tells the compiler that the function does not write through `src`. Combined with `restrict` on the destination, this gives the compiler complete information: the source is read-only, the destination is written, and they do not overlap. That is the ideal scenario for vectorisation.

Second, `const` prevents accidental self-aliasing bugs. If you accidentally write through a pointer you intended to be read-only, `const` catches it at compile time rather than producing a silently wrong (and hard-to-diagnose) result.

In the divide-and-conquer transpose code we wrote together in class, the source pointer was declared `const double *src` from the start. At the time, this seemed like nothing more than good practice — a defensive coding habit. It turned out to also be an optimisation: it simplified the alias analysis for the inner-loop base case and contributed to the compiler's willingness to vectorise. Clean code pays dividends you did not plan for.

20.5. The `-fstrict-aliasing` flag

There is a second, orthogonal mechanism that helps the compiler with aliasing, and it operates at the *type* level rather than the pointer level.

The C and C++ standards contain a rule called the **strict aliasing rule**: an object of type `T` may only be accessed through a pointer of type `T`, a pointer to a compatible type, a `char *` or `unsigned char *`, or (in C) a pointer to a `union` that contains `T`. Accessing a `double` through an `int *` (which is called *type punning*) is undefined behaviour under this rule.

Why does this matter for optimisation? Because with strict aliasing, the compiler can assume that a `double *` and an `int *` never point to the same memory. This dramatically simplifies alias analysis: when a function operates on arrays of different types, the compiler can freely reorder loads and stores across types without runtime aliasing checks.

GCC enables strict aliasing by default at `-O2` and higher. The flag `-fstrict-aliasing` turns it on explicitly; `-fno-strict-aliasing` turns it off. The latter is sometimes needed in legacy code that uses type punning through pointer casts — a practice that was historically common (especially in networking code that casts `char *` buffers to struct pointers) but is technically undefined.

In terms of what this means for you as a programmer:

At `-O2` or `-O3`, strict aliasing is already on. You are already benefiting from it — and if your code violates the strict aliasing rule, you may already have latent bugs that surface unpredictably across compilers and optimisation levels.

Never cast between unrelated pointer types for the purpose of reinterpretation. If you need to view the bytes of a `double` as an integer (a common trick in hashing or bit-manipulation), use `memcpy`:

```
double x = 3.14;
uint64_t bits;
memcpy(&bits, &x, sizeof(bits));   // correct, defined behaviour
```

Any modern compiler at `-O2` will optimise this `memcpy` into a single register move — there is no runtime cost. The cast-based alternative:

```
uint64_t bits = *(uint64_t *)&x;   // UNDEFINED BEHAVIOUR under strict aliasing
```

works on many compilers by accident, but is not guaranteed to work and *will* break under some optimisation passes or some compiler versions. It is not worth the risk.

If you work with C code that was originally written before C99, or with code that has been ported from environments where strict aliasing was not enforced (older GCC with `-fno-strict-aliasing`, MSVC which historically did not enforce it), you may encounter mysterious bugs when compiling at `-O2`/`-O3`. A common symptom: the code "works" at `-O0` but produces wrong results at `-O2`. Before blaming the compiler, check for type-punning through pointer casts. This is almost always the cause.

20.6. Summary: what you should do

The practical recommendations, in order of how much bang-for-buck they offer:

1. **Use separate allocations for separate arrays.** Two `malloc` calls cost nothing extra, and they give the compiler's alias oracle the information it needs for free.
2. **Use `restrict` on function parameters** whenever you know the pointers do not alias — which, again, is almost always. This is the single most effective thing you can do to help the compiler vectorise loop bodies across function boundaries.
3. **Use `const` on pointers that are not written through.** It communicates intent, prevents bugs, and sometimes helps alias analysis.
4. **Leave `-fstrict-aliasing` on** (it is on by default at `-O2`+). Do not type-pun through pointer casts; use `memcpy` instead.
5. **Check the assembly (`-S`) when in doubt.** If you expect a loop to be vectorised and it is not, aliasing is the first suspect. Look for scalar load/store sequences where you expected SIMD, and check whether `restrict` or separate allocations resolve it.

And above all: do not dismiss these issues as "compiler quirks" or "micro-optimisation trivia". We have seen, empirically, that aliasing decisions can produce a 5× difference in instruction count without changing a single byte of the actual computation. That is not a quirk; it is a structural feature of how compilers work, and understanding it is part of being a competent HPC programmer.

Part III — Prefetching

21. The latency problem, revisited

We have established that cache misses to DRAM cost 150–300 cycles, and that for irregular access patterns (tree traversals, hash lookups, sparse matrix operations) these misses dominate the runtime. The cache hierarchy handles regular sequential access well, but it cannot solve the fundamental problem: the data must exist in some cache level *before* the CPU needs it, or the core stalls.

Prefetching is the technique of issuing a memory request *before* the data is needed, so that by the time the load instruction executes, the data is already in cache. The latency has not been reduced — DRAM still takes 200 cycles — but it has been *overlapped* with computation.

More precisely: a prefetch is a memory read that populates a cache line *without stalling the issuing instruction*. It is fire-and-forget. If the prefetch arrives in time, the subsequent load completes in L1 latency (~4 cycles); if it arrives late, the load still stalls, but for less time; if it arrives too early, the line may have been evicted before use, wasting bandwidth and gaining nothing.

22. The prefetch distance

The key quantity is D, the number of useful computing cycles between the prefetch and the load. To fully hide a DRAM latency of L_mem cycles in a loop body that takes T cycles per iteration:

$D \approx L_mem / T$

For a simple arithmetic loop with $T \approx 4$ cycles and $L_mem \approx 200$ cycles, $D \approx 50$ iterations of lookahead. For a more expensive loop body, D shrinks; for a very cheap one, it can be impractically large.

This formula is worth memorising. It explains at once why explicit prefetching is fragile (both L_mem and T are hardware- and code-dependent) and why hardware prefetching works so well for regular patterns (the hardware can measure T and L_mem dynamically and self-tune D).

In other words: explicit prefetching pays off most when the loop body is computationally cheap and the access pattern is irregular. If the loop body is expensive, the prefetch distance is small and OoO execution may hide the latency without any prefetching at all. If the access pattern is regular, the hardware prefetcher handles it.

23. Three mechanisms for prefetching

Hardware prefetcher. A dedicated unit inside the CPU that monitors cache miss addresses and detects regular patterns: sequential scans, constant strides, and (on some architectures) irregular but repeated strides. It proactively issues loads for the addresses it predicts will be needed next. Remarkably effective for regular access patterns; for irregular or data-dependent patterns, it fails completely. On Intel, the prefetcher can be tuned or disabled per-core via MSR 0x1A4; on AMD, through `msr-tools` or `likwid-features`.

Software prefetcher. Explicit machine instructions — `PREFETCHT0` / `T1` / `T2` / `NTA` on x86, `__builtin_prefetch()` in GCC — inserted by the programmer or compiler. Can in principle handle arbitrary patterns, since the programmer computes the address to prefetch. Its weakness is that placing the instruction at the right point is difficult, system-dependent, and fragile across compiler versions.

OoO execution as an implicit prefetcher. The out-of-order engine can have multiple independent loads in flight simultaneously, creating Memory-Level Parallelism (MLP). If the reorder buffer contains several loads to different cache lines, their DRAM latencies overlap. This is not prefetching in the strict sense, but it has the same effect; code that enables high MLP — through loop unrolling, ILP-friendly structure, absence of false dependencies — benefits from this automatically. The binding constraint for MLP is not the ROB size but the number of Miss Status Holding Registers (MSHRs), typically 10–20 per core, which limits outstanding cache misses the CPU can track simultaneously.

24. Explicit prefetching: a binary search example

Binary search on a sorted array is a textbook case of an irregular access pattern: at each step, you access the midpoint of the remaining range, which is unpredictable for the hardware prefetcher. On large arrays, virtually every comparison is a DRAM hit.

The idea: after computing `mid`, immediately prefetch the midpoints of both possible sub-ranges — the ones we will need regardless of which branch is taken:

```
while (low <= high) {
    mid = (low + high) / 2;
    __builtin_prefetch(&data[(mid + 1 + high) / 2], 0, 0);
    __builtin_prefetch(&data[(low + mid - 1) / 2], 0, 0);
    if (data[mid] < Key)        low = mid + 1;
    else if (data[mid] > Key)  high = mid - 1;
    else return mid;
}
```

The `__builtin_prefetch(addr, rw, locality)` arguments: `0` = read, `0` = non-temporal (NTA, load directly to L1, do not pollute higher levels since we will use it only once). One could also prefetch the next *two* levels (4 possible midpoints) for deeper lookahead.

The results, measured with PAPI on an older 10th-gen Intel:

Without prefetching: 71% of loads hit DRAM, 24% hit the LFB (Line Fill Buffer — data in transit), ~4% L3.
With prefetching: 68% LFB, 27% L1, ~3% DRAM.

The LFB hits are the "arrived late" scenario: the prefetch was issued but the load caught up with it before DRAM responded. This is still cheaper than a full DRAM hit (~10–30 cycles vs ~50+), which is why the speedup is 1.6× rather than nothing. The 27% L1 hits are the cases where the prefetch worked perfectly.

The lesson: even a suboptimal prefetch distance is better than nothing, but the data shows we have not tuned D optimally. This could be turned into an exercise: adjust the prefetch distance and observe the L1/LFB ratio.

25. Prefetching by preloading

A safer and more portable alternative to explicit prefetch instructions is *preloading*: reading future data into a local variable early, so that the cache miss fires while the CPU is busy with other work.

```
elem a = elem[0];
for (int i = 0; i < 4*N_4; i += 4) {
    elem e = elem[i+4];   // non-blocking miss for next iteration
    elem b = elem[i+1];   // likely cache-hit (same line as elem[i])
    elem c = elem[i+2];   // likely cache-hit
    elem d = elem[i+3];   // likely cache-hit
    Elaborate(a);
    Elaborate(b);
    Elaborate(c);
    Elaborate(d);
    a = e;                 // carry the preloaded value to next iteration
}
```

The idea: `e = elem[i+4]` fires a load for the *next* group of elements while the CPU is busy elaborating the *current* group. If the loop body (the four `Elaborate()` calls) takes enough cycles to cover the DRAM latency, `e` will be sitting in L1 by the time it is used as `a` in the next iteration.

There is a practical complication: the compiler, at `-O3`, is free to reorder loads. It may move the load of `elem[i+4]` to where it thinks is optimal — which might be right before it is used, defeating the purpose. In practice, though, most compilers with `-O3` try to place loads as early as possible precisely to maximise the latency-hiding window, so the intent usually survives. But it is worth checking with `-S`.

26. The bottom line on prefetching

For regular access patterns, the hardware prefetcher is almost always sufficient. Your job is to *not defeat it*: use sequential or short-stride access, avoid pointer chasing, and unroll enough to expose ILP.

For irregular patterns, explicit prefetching (via `__builtin_prefetch` or preloading) can help, but it is fragile and system-dependent. The prefetch distance formula D ≈ L_mem/T gives you the ballpark; PAPI counters (specifically `PAPI_L1_DCM`, `PAPI_PRF_DM`, and the `perf mem` memory-access breakdown) let you verify whether the prefetches are arriving in time.

In all cases, the structural approach — choosing algorithms with regular access patterns, laying out data to match the traversal order — is more robust than patching with prefetch instructions after the fact.

Part IV — Branches

27. Conditional execution: two mechanisms

Whenever the next instruction to execute depends on the outcome of a test, we have a conditional execution. At the machine level, modern x86 provides two distinct mechanisms:

Control-flow transfer (jumps). The classic approach: evaluate a condition, set flags in the EFLAGS register (ZF, SF, OF, CF), and execute a conditional jump (`je`, `jne`, `jg`, `jl`, ...) that either continues sequentially or jumps to a different code address. If the jump is taken, the instruction stream changes discontinuously.

Data-flow transfer (cmov). The alternative: compute *both* outcomes, then use a conditional move (`cmov`) to select the correct result without any jump. No discontinuity in the instruction stream.

The flags register deserves a brief note: the Zero Flag (ZF) is set when the last arithmetic or logical operation produced zero; the Sign Flag (SF) records the sign; the Carry Flag (CF) signals unsigned overflow; the Overflow Flag (OF) signals signed overflow. The conditional jump instructions test combinations of these flags. This is the low-level mechanism behind every `if`, `while`, `for`, and `switch` in your high-level code.

28. How a `for` loop looks in assembly

A simple loop:

```
for (int i = 0; i < 10; i++)
    array[i] = 0;
```

Compiles (at `-O0`) to something like:

```
.L2:
    movq $0, (%rax)         ; array[i] = 0
    addq $8, %rax           ; i++ (advance pointer by sizeof(int) or sizeof(double))
    cmpq %rbp, %rax         ; compare i with upper bound
    jne .L2                  ; if not equal, jump back
```

The `cmpq` computes `rax - rbp` and sets the flags; `jne` tests ZF and jumps if it is not set (i.e., if they are not equal). Until the loop terminates, the jump is always taken (backward jump to `.L2`); when it terminates, execution falls through.

An `if/else` is similar but with a forward jump to the `else` branch. The *true* branch is placed immediately after the condition test (fall-through), and the *false* branch is reached by a forward jump. This layout matters for code locality: the fall-through path is in the same cache line, while the jump target may not be.

Remind: you can give the compiler a hint about which branch is likely:

```
if (__builtin_expect(condition, 1)) { /* likely path */ }
// or in C++20:
if (condition) [[likely]] { /* ... */ }
```

This tells the compiler to place the likely path as the fall-through and the unlikely path after the jump. It also informs the static branch predictor (used for cold branches, before the dynamic predictor has learned the pattern).

29. Branch prediction and the pipeline

Modern CPUs achieve their performance through deep pipelines and out-of-order execution: dozens of stages between fetch and retire, with hundreds of instructions simultaneously in flight (the ROB on Intel Alder Lake holds 512 entries; AMD Zen 4, 320). This works brilliantly — as long as the CPU knows *which* instructions to fetch next.

A conditional branch is a problem: the CPU must decide whether the branch is taken *before* it has been executed. If it guesses correctly, the pipeline stays full and there is no penalty. If it guesses wrong, all the instructions that were speculatively fetched and partially executed along the wrong path must be flushed, and the pipeline restarts from the correct target. **The misprediction penalty is typically 10–30 cycles**, depending on the pipeline depth.

Modern branch predictors are remarkably sophisticated — they maintain per-branch history tables, global history patterns, and loop counters. For well-structured code, they achieve 95–99% accuracy. But even 95% accuracy can be devastating.

Worked example. Suppose you have 140 instructions in flight and 1 out of every 7 is a branch. That is 20 branches in the pipeline. With 95% prediction accuracy per branch, the probability that *none* of them is mispredicted is 0.95^20 ≈ 0.36 — only 36% of the time does the pipeline survive without a flush! With 90% accuracy, it drops to 0.90^20 ≈ 0.12. The point: even a very good predictor cannot save you if your code has many branches; the exponential accumulation of small misprediction probabilities is brutal.

30. The cmov instruction

The conditional move instruction is the hardware's answer to the pipeline problem for simple conditionals. Instead of:

```
cmp a, b
jg .true_branch
; false path: result = a - b
jmp .end
.true_branch:
; true path: result = a + b
.end:
```

The CPU computes *both* outcomes and selects:

```
mov edx, eax         ; edx = a
sub edx, ebx         ; edx = a - b
lea ecx, [eax+ebx]   ; ecx = a + b
cmp eax, ebx         ; compare a, b
cmovg edx, ecx       ; if a > b, edx = ecx (overwrite with a+b)
```

No jump, no branch, no pipeline flush. The execution flow is perfectly linear and predictable.

At `-O3`, the compiler will typically emit `cmov` for ternary expressions like `result = (a > b) ? (a + b) : (a - b)`. At `-O0`, it will emit the jump-based version.

However, `cmov` has a limitation: **both sides must be computable without side effects**. If one branch involves a memory access that might fault (division by zero, access to an unmapped page), a function call, or any observable side effect, the CPU cannot speculatively evaluate it. In such cases, the compiler must fall back to a conditional jump.

There is also a performance subtlety. If the branch is *highly predictable* (say, 99.5% one-way), the `cmov` approach computes the unnecessary side for nothing, while a correctly-predicted branch would simply skip it. In such cases, `cmov` is slower. The compiler's heuristic trades off these factors; when in doubt, check with `-S`.

31. Conditionals inside loops: why they are so expensive

A single `if` statement outside a loop is benign — the branch predictor handles it. But a conditional inside a tight inner loop is a different matter entirely: the branch is evaluated at every iteration, and if the outcome depends on unpredictable data, the predictor is wrong roughly half the time.

The classic demonstration (see `examples_on_branches/conditional.c`):

```
// Random data
for (int i = 0; i < SIZE; i++)
    data[i] = rand() % TOP;

// Conditional accumulation
for (int i = 0; i < SIZE; i++) {
    if (data[i] > PIVOT)
        sum += data[i];
}
```

With random data, the branch is taken about half the time with no pattern. The branch predictor guesses wrong roughly 50% of the time. For 500 million elements, this means ~250 million mispredictions, each costing 10–30 cycles. The performance data shows: same number of instructions per element, but nearly 3× more cycles per element compared to predictable data, because the IPC drops from ~2.7 to ~1.

If you sort the array first (`qsort`), the branch becomes perfectly predictable: all elements below PIVOT come first (branch not taken), then all above (branch taken). The misprediction count drops to essentially zero. Of course, sorting adds its own cost — this is not a practical optimisation, just a demonstration of the branch predictor's impact.

32. Strategies for eliminating conditionals

The practical strategies, in order of preference:

[1] Branchless arithmetic / predication. The general principle: compute both outcomes unconditionally, then select. No jump, linear flow.

```
// Instead of:
if (data[i] > 0)
    result += data[i] * factor_a;
else
    result += data[i] * factor_b;

// Write:
int mask = -(data[i] > 0);     // all-ones if true, zero if false
int factor = (factor_a & mask) | (factor_b & ~mask);
result += data[i] * factor;
```

Or, more readably (and the compiler will typically emit `cmov` for this at `-O3`):

```
result += data[i] * (data[i] > 0 ? factor_a : factor_b);
```

The applicability condition is strict: both branches must produce a *single scalar value*, with no side effects. The moment one branch calls `sin()` and the other calls `log()`, this does not apply.

There is also the cost trade-off. Branchless code computes both sides unconditionally. If the branch would have been highly predictable and one side is cheap while the other is expensive, you are wasting work. Branchless wins when the branch is *unpredictable*; it can lose when the branch is predictable and the outcomes have asymmetric cost.

Also: loops with conditionals cannot be vectorised by the compiler unless the conditional can be translated into a mask. The branchless formulation using masks is often *exactly* what the compiler needs to vectorise the entire loop.

[2] Data partitioning. If the data is already in memory and the per-element processing is expensive enough to amortize the partitioning cost, separate the data into subsets and run specialized (branchless, vectorisable) loops over each subset. A real trade-off, but for large data sets it can win substantially.

[3] Restructure the data. If the conditional discriminates categories of elements in a struct, the problem may be the data layout. Switching from Array of Structures (AoS) to Structure of Arrays (SoA) can eliminate the conditional entirely.

[4] Table-driven dispatch. Use function pointers indexed by the condition:

```
typedef void (*func_t)(double *, int);
func_t functions[] = { func_A, func_B, func_C };
for (int i = 0; i < N; i++) {
    int idx = select(data[i]);
    functions[idx](&result, data[i]);
}
```

This replaces an unpredictable conditional branch with an *indirect* branch, which has its own predictor (the Indirect Branch Target Buffer). Whether this is better depends on how many targets there are and how regular the pattern is. It is also likely that the individual functions will be less optimised than inlined code (no inter-procedural optimisation, no vectorisation across iterations). This is a last-resort strategy.

[5] Re-design the algorithm and the data. When you face a deeply nested, data-dependent if-forest that is not reducible to simple arithmetic selection, the algorithm itself needs rethinking. The if-forest is a *symptom*: it signals that the algorithm mixes heterogeneous computations in a single traversal because that was the natural *human* way to write it, not the most efficient structure for the hardware. The computation is conceptually operating on homogeneous subsets, but the data layout or algorithm structure forces it to re-examine which subset every element belongs to at every iteration. Fixing that — at whichever level is appropriate — is almost always more effective than patching the branch behaviour after the fact.

33. Examples of code restructuring

Example 1: conditional accumulation.

```
// Branchy:
for (int i = 0; i < SIZE; i++)
    if (data[i] > PIVOT) sum += data[i];

// Branchless:
for (int i = 0; i < SIZE; i++)
    sum += (data[i] > PIVOT) * data[i];
```

The expression `(data[i] > PIVOT)` evaluates to 0 or 1; multiplying by `data[i]` produces 0 or `data[i]`. No branch. The compiler at `-O3 -march=native` will vectorise this (using SIMD compare + blend instructions), processing 8 integers or 4 doubles per cycle.

Example 2: sorting two arrays.

```
// Branchy:
for (int i = 0; i < SIZE; i++)
    if (A[i] < B[i]) { int t = B[i]; B[i] = A[i]; A[i] = t; }

// Branchless:
for (int i = 0; i < SIZE; i++) {
    int min = A[i] > B[i] ? B[i] : A[i];
    int max = A[i] >= B[i] ? A[i] : B[i];
    A[i] = max;
    B[i] = min;
}
```

Performance data: same number of instructions per element, but the branchless version has stable cycles-per-element regardless of whether the data is predictable or random (~3 IPC in both cases), while the branchy version drops from ~2.7 IPC (predictable) to ~1 IPC (random).

Example 3: matrix initialisation with position-dependent values.

```
// Branchy:
for (int j = 0; j < N; j++)
    for (int i = 0; i < M; i++) {
        if (i > j)       matrix[j][i] = 1.0;
        else if (i < j)  matrix[j][i] = -1.0;
        else              matrix[j][i] = 0.0;
    }

// No branches at all:
for (int j = 0; j < N; j++) {
    int i;
    for (i = 0; i < j; i++)        matrix[j][i] = -1.0;
    matrix[j][i] = 0.0;
    for (i = j + 1; i < M; i++)    matrix[j][i] = 1.0;
}
```

Three branch-free loops, each doing a single constant assignment. The condition has been absorbed into the loop bounds. This is the kind of restructuring that the compiler *cannot* do for you — it requires understanding the semantics of what you are computing.

34. Profile-Guided Optimisation (PGO)

When you genuinely do not know which branches are likely at compile time, PGO is the systematic solution:

1. Compile with `-fprofile-generate` added to your optimisation flags.
2. Run the executable with *representative* input data. The runtime logs branch outcomes, loop trip counts, and call frequencies.
3. Recompile with `-fprofile-use`. The compiler reads the logged profile and reshuffles the code: likely branches become fall-through, hot loops are unrolled more aggressively, cold paths are moved to separate pages.

PGO is particularly effective for code with complex control flow where static heuristics fail. The overhead is: one extra compilation cycle and the need for representative data. For production HPC codes, this is often worth it.

Part V — Putting It All Together

35. A word of caution

It is easy to get lost in micro-optimisation — spending hours chasing a 2% improvement in a single loop while ignoring a 10× improvement available from a better algorithm or a smarter data layout. The most effective optimisations are, by far:

1. **The algorithm.** An O(N log N) algorithm with terrible cache behaviour will still beat an O(N^2) algorithm with perfect cache behaviour for large enough N.
2. **The data model.** How you lay out your data in memory determines everything downstream: cache hit rates, vectorisation opportunities, branch patterns, prefetching effectiveness.
3. **Code quality.** Clean, readable code is not just a human virtue — it also helps the compiler. Code with clear data flow, minimal aliasing, well-scoped variables, and no unnecessary control-flow complexity is code that the compiler can reason about and optimise effectively.

Everything else — loop unrolling, branchless tricks, explicit prefetching, SIMD intrinsics — is important, but it is second-order. Get the algorithm and data layout right first; then tune.

36. Reference codes and exercises

The following codes are in the course repository under `SCO/`:

- `examples_on_cache/memory_mountain/mountain.c` — the memory mountain benchmark.
- `examples_on_cache/cache_size.c` — measuring the cache-miss signature.
- `examples_on_cache/matrix_transpose/` — all variants of matrix transposition (naive, swapped, padded, tiled, cache-oblivious).
- `examples_on_pipelines/matrix_multiplication/` — naive, loop-swapped, and tiled matrix multiplication, instrumented with PAPI.
- `examples_on_pipelines/reduction/` — all unrolling variants of the array reduction.
- `examples_on_branches/conditional.c` — the conditional-accumulation example with branchy and branchless variants.
- `examples/prefetching/explicit_prefetching/` — the binary search with explicit `__builtin_prefetch`.
- `examples/prefetching/preload_prefetching/` — prefetching by preloading.

All codes are instrumented with PAPI to give you hardware counter data (L1 misses, branch mispredictions, instructions per cycle, cycles per element). At this stage in the course, treat the PAPI instrumentation as a black box — we will discuss hardware counters in detail later.

Suggested exercises:

- Run the memory mountain on your laptop. Identify the L1, L2, L3, and DRAM plateaus. How do they

compare to the nominal cache sizes of your CPU?

- Measure the matrix transpose bandwidth for sizes from 100 to 10000. Can you identify the three regimes? Can you spot the power-of-two anomalies?
- Compare the tiled and naive matrix multiplication for N = 2000. What block size gives the best performance? What happens if you use a power-of-two block size?
- In the conditional-accumulation example, compare the branchy and branchless versions with random and sorted data. Verify the IPC and misprediction data.
- In the binary search example, modify the prefetch distance (prefetch further ahead) and see if the L1 hit rate improves. How far ahead do you need to prefetch to see the best L1/LFB ratio?

Next: OpenMP — multi-threading for shared-memory parallelism.